

Chapter 1

Designing a Large-Scale Social Feed Ranking System

A visual, alternatives-driven ML system design case study

PRIMARY REFERENCE
Study foundation
This chapter began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, and interpretations are the author's own.

Chapter at a glance

Design area	Current direction
Scale	500M DAU working assumption; ~200 ms ranking request
Objective	Predict several user behaviors and construct a serving score
Retrieval	Two-tower / ANN, social graph, and other candidate sources
Final model	Shared-bottom multi-task DNN
Validation	Offline evaluation, A/B test, then canary rollout
Scope boundary	Advertising monetization excluded

1. Case map and minimum scope

Design the smallest credible system that can rank a social feed under large-scale latency and reliability constraints. Separate retrieval from final ranking, and add model granularity only where the product objective needs it.

Figure 1.1 - Ranking-system design map.

CONFIRMED INTERPRETATION
Scale assumptions are challengeable
The 500 million DAU and roughly 200 ms latency values are working assumptions used to force large-scale design reasoning. They are not claims about a specific company.

OPEN DESIGN DISCUSSION
What is the minimum sufficient scope?
Main product action: _____
Scale and latency assumption: _____
Explicitly out of scope: _____
Condition that expands scope: _____

2. Product objective and extensible labels

A ranking system should begin with the behavior it is intended to improve. One impression may generate several Boolean outcomes, allowing the same training example to support multiple product objectives.

Figure 1.2 - One impression can generate several extensible labels.

Label type	Example	Open product decision
Positive engagement	like, comment, share	Which behaviors represent meaningful value?
Timing	interaction within T	What attribution window is appropriate?
Dwell	dwell time at or above threshold	What threshold separates useful consumption?
Negative feedback	hide, block	How strongly should it reduce the serving score?

FROM THE ORIGINAL NOTES

Extensible label set

New product objectives can become new labels and, in a multi-task model, new heads. The notes use 1 and 0 as positive and negative outcomes for each selected label.

3. Metrics by stage

The notes survey metrics rather than selecting one universal answer. The correct metric depends on the system stage, label type, ranking depth, and product question.

Figure 1.3 - Stage-specific metric selection.

Metric	Best use	Main caution
Precision@K	Visible top-K quality	Does not fully describe order inside K
Recall@K	Candidate-generation coverage	May look small when positives greatly exceed K
mAP	Binary relevance across relevant ranks	Treats relevance as binary
MRR	First relevant result matters most	Ignores later relevant results
nDCG@K	Graded relevance and ordering	Requires a gain definition
PR-AUC	Rare positive behavior	Not a ranked-list metric
ROC-AUC	Broad discrimination	May hide top-K product weakness

STARTING RECOMMENDATION

Use a metric suite

Use Recall@K for retrieval, nDCG@K or Precision@K for final ranking, PR-AUC for rare behavior heads, and online product metrics for the final decision.

4. Model alternatives and their roles

Collaborative filtering, content-based models, two-tower retrieval, and a multi-task ranker are not mutually exclusive competitors. They solve different parts of the system.

Figure 1.4 - Model alternatives arranged as a staged production design.

Option	Best when	Main benefit	Main cost
Collaborative filtering	Interaction history is strong	Credible baseline	Cold start
Content-based	New users or content need support	Uses attributes and modality features	May miss community interaction patterns
Two-tower	Large-scale retrieval is required	Scalable independent encoding	Less expressive pair interaction
Shared-bottom multi-task	Final ranking needs granular behavior scores	Shared learning plus task specialization	Higher serving and tuning complexity

CONFIRMED INTERPRETATION

Two-tower is broader than collaborative filtering

A two-tower model using only user-ID and item-ID embeddings is CF-like. Adding demographics, history, text, metadata, and context makes it a richer hybrid architecture.

PERSONAL LEARNING QUESTION

Shared-bottom versus Mixture of Experts

The drawn model resembles MoE only conceptually through shared and specialized computation. It is not an MoE because it has no expert routing or gating mechanism.

5. Shared-bottom multi-task ranking

The preferred final ranker shares a common DNN representation and uses smaller task-specific heads to predict several behaviors.

Figure 1.5 - Multi-task architecture and the distinction between training and serving weights.

Feature group	Examples	Role
User	ID, location, language, history	Viewer representation
Content	text, image, video, topic, author	Post representation
Historical engagement	views, likes, comments, dwell	Longer-term preference
User-author relationship	friendship, duration, strength, mutual links	Relationship-specific relevance
Context	time, device, request state	Request-specific behavior

CONFIRMED INTERPRETATION

Relationship features are model inputs

Friendship is not a separate prediction head and is not manually added after model scoring. It is a family of user-author input features.

MINIMAL-SCOPE RULE

Do not copy serving weights into the loss

Training weights λ_i balance learning and stay nonnegative. Serving weights w_i express product utility and may be negative for hide or block.

6. Data, features, and freshness

Document feature meaning and availability separately. Historical does not automatically mean batch, and aggregated does not automatically mean stale.

Property	Meaning	Example
Structured	Categorical or numeric fields	age, language, IDs, timestamps
Unstructured	Text, image, audio, or video	BERT, ViT, ResNet, or CLIP embeddings
Aggregated	Summarizes multiple events	seven-day engagement rate
Delayed	Arrives after asynchronous processing	hourly recomputed historical feature
Online	Available at request time	device and current request context

CONFIRMED INTERPRETATION

Aggregated and delayed are different properties

A feature can be aggregated and delayed at the same time. Freshness should be documented independently from feature semantics.

7. Training and loss weighting

For task i , the total training objective is the weighted sum of head-specific losses: $L_{\text{total}} = \sum_i \lambda_i L_i$.

- Head-specific parameters receive gradients from their own task.
- Shared layers receive the weighted combined influence of all tasks.
- Loss weights may consider label frequency, loss magnitude, task difficulty, gradient interference, and final ranking quality.
- The dwell threshold and attribution windows remain product decisions.

OPEN DESIGN DISCUSSION

There is no universal formula for λ_i

Business priority matters, but the training weights should also be validated through per-task and final-ranking performance.

8. Candidate generation and final ranking

At very large scale, the richer ranker should score only a manageable candidate set. Several retrieval sources can contribute candidates before one final ranking stage.

Figure 1.6 - Candidate generation, deduplication, feature hydration, and final ranking.

CONFIRMED INTERPRETATION

Multiple retrieval hits are informative

The system should deduplicate by content ID but retain source indicators. Agreement between retrieval sources can itself be useful ranking evidence.

OPEN DESIGN DISCUSSION

How should candidate budgets be allocated?

ANN pool size: _____

Graph pool size: _____

Other source pool: _____

Deduplication and source-feature policy: _____

9. Indexing service boundary

The indexing service converts candidate representations into searchable structures. It hides the exact retrieval technique behind a stable service interface.

Figure 1.7 - Indexing service and optional compression techniques.

Component	Responsibility
Feature store	Reusable model features
Indexing service	Build and version searchable representations
Candidate-generation service	Query one or more indexes
Ranking service	Score enriched, deduplicated candidates

FROM THE ORIGINAL NOTES

Optional vector compression

Vector quantization and product quantization can reduce memory and accelerate approximate retrieval. They remain optional techniques inside the indexing service.

10. Online validation and deployment

Offline performance is necessary but not sufficient. The new model should be tested online, then deployed gradually with rollback available.

Figure 1.8 - A/B testing, canary rollout, monitoring, and retraining.

Method	Question answered
Offline evaluation	Does the model satisfy model-quality gates?
A/B test	Does it improve online product metrics?
Canary rollout	Can it operate safely at increasing production scale?
Rollback	Can traffic return quickly to a stable system?

CONFIRMED INTERPRETATION

The 5% / 95% split is an A/B test

After experimental performance and robustness are demonstrated, the canary rollout is a separate staged deployment step.

OPEN DESIGN DISCUSSION

What permits wider deployment?

Offline gate: _____

A/B primary metric: _____

Canary health metrics: _____

Rollback condition: _____

11. Monitoring and continual learning

Model quality can decline as user behavior, content supply, and platform conditions change. Monitoring should connect model quality, data quality, and system health.

Monitor	Examples	Possible response
Model quality	ranking metrics, head metrics	investigate or retrain
Data drift	feature and label distributions	refresh data or features
Product behavior	engagement and satisfaction	review objectives or weights
System health	latency and availability	rollback or scale infrastructure

FROM THE ORIGINAL NOTES

Continual-learning options

Regularization, adapters or new layers, and replaying some older data may reduce catastrophic forgetting. Replaying older data can distort the current time distribution.

12. Reference architecture and decision summary

One defensible design under the current assumptions is a hybrid system: reusable features and indexes are prepared asynchronously, while online retrieval, merging, feature hydration, and multi-task ranking meet the request-time latency target.

Figure 1.9 - One defensible large-scale feed-ranking architecture.

STARTING RECOMMENDATION

Hybrid retrieval plus multi-task ranking
Use two-tower or ANN retrieval and graph-based sources to form a candidate set, then apply the richer shared-bottom multi-task model to the merged and hydrated candidates.

Design area	Starting choice	Switch when
Objective	Multi-behavior prediction	One scalar target is demonstrably sufficient
Retrieval	ANN / two-tower plus graph sources	A simpler source meets coverage and latency
Final model	Shared-bottom multi-task DNN	A simpler baseline performs similarly
Metrics	Stage-specific suite	One metric reliably represents all decisions
Indexing	ANN abstraction with optional VQ/PQ	Exact search fits scale and latency
Validation	A/B test, then canary	Product risk requires another method